



Programação Orientada a Objetos

Introdução ao Java

Maurício da Motta Braga
mmb2@cesar.school





*"Java is C++ without the
guns, knives, and
clubs."*

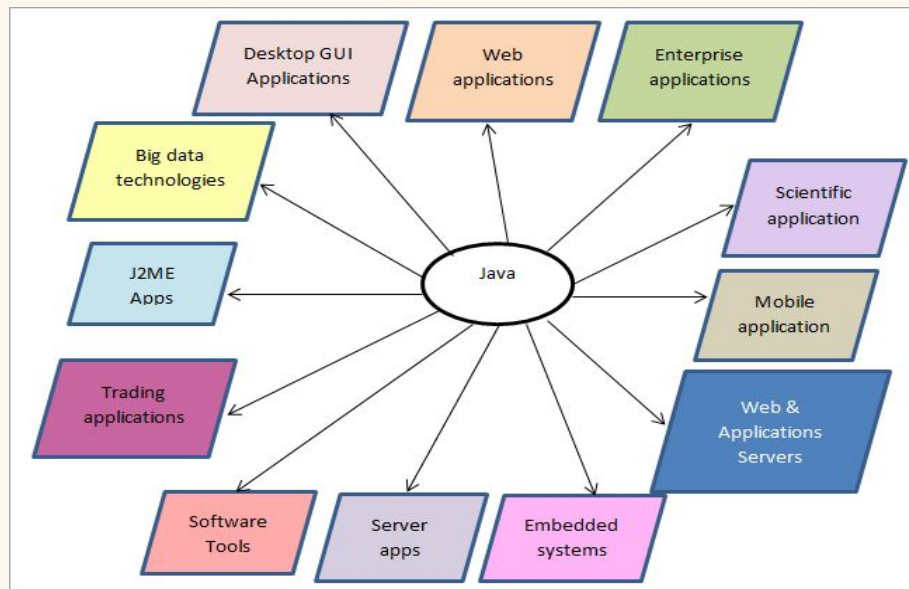
James Gosling.

Por que aprender Java?

Uma das linguagens mais usadas no mundo!!



Serve a diferentes propósitos!!



Milhões de recursos compatíveis



...e muito mais!

Por que JAVA?



Fortemente tipada

Orientada a objetos

Alocação dinâmica
de variáveis (de
memória)

Desalocação
automática de
memória (garbage
collector)

Não gera código
executável nativo, e
sim um runtime
interpretável

É naturalmente
multiplataforma
(write once, run
anywhere)

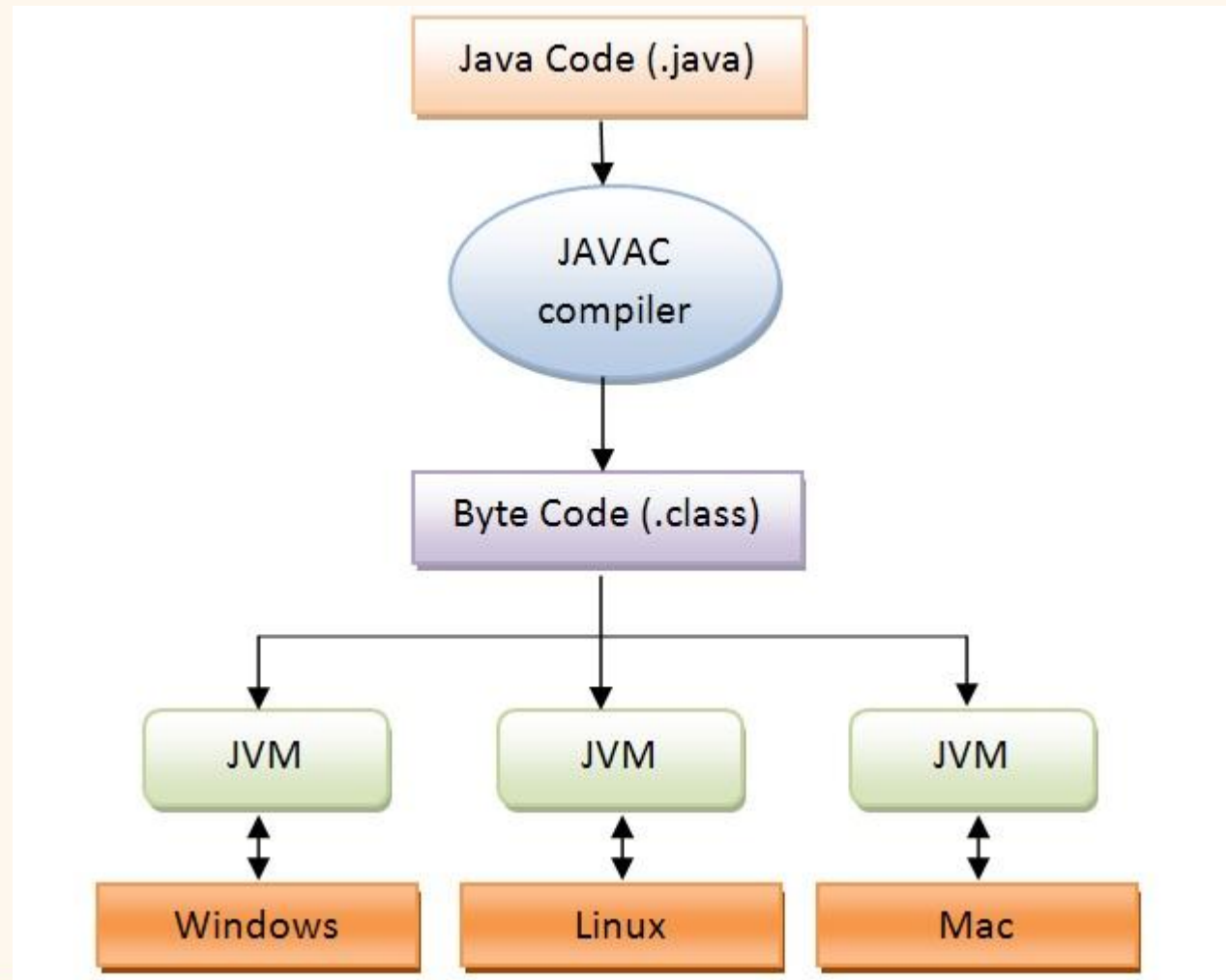
Existem literalmente
milhões de recursos
de terceiros
desenvolvidos em e
para JAVA

Na prática, JAVA é
uma grande e
consistente
plataforma de
desenvolvimento

“DNA” do C e do C++

Por que Java?

JVM - JAVA Virtual Machine



Java



- ▶ No JAVA, toda e qualquer estrutura de um programa é uma classe
- ▶ Um programa em JAVA é composto por uma ou mais classes
- ▶ Como na linguagem C, em JAVA, a classe que representa o “ponto de partida” de um programa tem um “procedimento” (mais tarde vamos chamar procedimentos de métodos) com uma assinatura específica

```
pse ide
Run Window Help
module-info.java MainBasic.java ×
1 package teste.aula;
2
3 public class MainBasic {
4
5     public static void main(String[] args) {
6         System.out.println("Hello JAVA!");
7     }
8 }
9
```

JCC - Java code convention

- ✓ Java é sensível ao contexto
 - ✓ **Maiúsculas são diferenciadas de minúsculas**
 - ✓ Nomes de classes começam com letra maiúscula (convenção)
 - ✓ Ex.: `public class Cliente`
 - ✓ Nomes de constantes são escritos com todas as letras maiúsculas (convenção), e, se for uma palavra composta, se coloca “_” .
 - ✓ Ex.: `final double SALARIO_MINIMO;`
 - ✓ Nomes de variáveis e de métodos começam com letra minúscula (convenção).
 - ✓ Ex.: `double numero;`
 - ✓ Para variáveis, métodos e classes, caso o nome seja uma palavra composta, não se coloca “_”. A primeira letra palavra seguinte é em maiúsculo (camelCase) (convenção)
 - ✓ Ex.: `int anoDePublicacao`
 - ✓ Ex.: `public class PessoaFisica`
-

Java – Tipos primitivos

- ▶ Em JAVA, quase todo tipo é definido por classes
- ▶ Mas existem tipos básicos (tipos primitivos) que não são classes

<i>Tipo</i>	<i>Tamanho (bits)</i>	<i>Valor Mínimo</i>	<i>Valor Máximo</i>	<i>Sem Sinal</i>
boolean	1	false	true	X
char	16	0	$2^{16} - 1$	X
byte	8	-2^7	$2^7 - 1$	
short	16	-2^{15}	$2^{15} - 1$	
int	32	-2^{31}	$2^{31} - 1$	
long	64	-2^{63}	$2^{63} - 1$	
float	32			
double	64			

Java - Comentários

```
public class Comentarios {  
    /*  
     * Este é um ponto de entrada de um programa JAVA.  
     * Esquema similar ao C.  
     * O args é um array de textos, cujo conteúdo  
     * é uma sequência de textos passados como  
     * parâmetros na linha de chamada para a  
     * execução do programa  
     */  
    public static void main(String[] args) {  
        // Comentários em linha simples  
        // Bla bla bla  
        /*  
         * Comentários em múltiplas linhas  
         * Bla  
         * bla  
         */  
    }  
}
```

Java – Variáveis e constantes

```
public class VariaveisConstantes {  
    public static void main(String[] args) {  
        // Definição de variáveis (tipo nome = valor inicial)  
        int a;  
        double b = 3.1;  
        boolean c = false;  
        long d = 210000000000L;  
        float f = 2.1F;  
        byte g;  
        short h;  
        char i = 'P';  
        final int VALOR_CONSTANTE = 33;  
    }  
}
```

Java – Variáveis e constantes

- ▶ Variáveis podem ser definidas em qualquer lugar de um programa JAVA, a qualquer tempo – vamos falar mais disso lá na frente
- ▶ Por enquanto, vamos definir nossas variáveis no início de cada programa
- ▶ A atribuição de um valor inicial pode ser feita na linha de definição da variável ou em outra linha subsequente

`<tipo> <nome da variável>;`

OU

`<tipo> <nome da variável> = <valor inicial>;`

```
int a;  
double salario = 2000.21;  
char letra2 = 'C';  
boolean eReal = true;  
float p = 5.0F;  
short simples211;  
byte b = -120;  
long h = 993298298322932L;
```

Operador de atribuição!!

Pode ser:

1. Uma constante
2. Uma variável
3. Uma expressão

Java - IO básico

```
import java.util.Scanner;
public class BasicoIO {
    private static final Scanner ENTRADA = new Scanner(System.in);
    public static void main(String[] args) {
        // Leitura de dados de vários tipos do teclado
        int a = ENTRADA.nextInt();
        double d = ENTRADA.nextDouble();
        boolean b = ENTRADA.nextBoolean();
        long la = ENTRADA.nextLong();
        /*
         * Em todos os casos acima, o fluxo de execução abre um
         * cursor no console e espera o usuário digitar um valor e
         * pressionar ENTER. Após o ENTER, o valor digitado, se
         * compatível com o tipo lido (nextInt, nextDouble, etc) é
         * atribuído à variável, que recebe o retorno do comando de leitura.
         */
        System.out.print(a); // imprime no console sem pular linha
        System.out.println(d); // imprime no console pulando linha
        System.out.println("OLÁ!!"); // imprime no console texto
    }
}
```

Prática

- ▶ Objetivos
 - ▶ Familiarizar-se com a criação de projetos em Java

Java - Operadores básicos

```
public class OperadoresAritmeticos {  
    public static void main(String[] args) {  
        int a = 2;  
        double d = 4.0;  
        double soma = a + d;  
        double subtracao = a - d;  
        double produto = a * d;  
        int divisaoInteira = a / 2;  
        double divisao = d / a;  
        int resto = 5 % a;  
        a++; // incremento unário  
        a--; // decremento unário  
        a += 2; // a = a + 2  
        a -= 2; // a = a - 2  
        a *= 2; // a = a * 2  
        a /= 2; // a = a / 2  
    }  
}
```

```
public class OperadoresLogicosRelacionais {  
    public static void main(String[] args) {  
        int a = 2;  
        double d = 3.1;  
        // || - OU  
        // && - E  
        // ! - NÃO  
        // == - igual a  
        // != - diferente de  
        // > - maior que  
        // >= maior ou igual a  
        // < - menor que  
        // <= menor ou igual a  
        boolean b1 = (a == d || a != 2);  
        boolean b2 = (a != 2 && !(a >= 2));  
        boolean b3 = !(a > d && d < 92.2);  
    }  
}
```

Promoção aritmética

- ▶ É possível realizar operações aritméticas com tipos de dados diferentes
- ▶ Mas...o JAVA, antes de efetuar uma operação aritmética, converte o operando “mais curto” para o tipo do operando “mais largo”
- ▶ Tanto é que o resultado da operação, deve ser armazenado em uma variável, do tipo “mais largo” encontrado na expressão àquela atribuída

Uma operação aritmética com operandos de tipos diferentes em mais detalhes...

- ▶ `int a = 3;`
- ▶ `float f = 2.0F;`
- ▶ `float z = a + f;`

`a + f -> 3 + 2.0F -> 3.0F + 2.0F -> z = 5.0F`

Biblioteca de funções matemáticas

// Os retornos obviamente se referem aos exemplos específicos!!

Math.abs(-1); // retorna 1, suporta qualquer expressão numérica

Math.pow(2, 3); // retorna 8, suporta quaisquer expressões numéricas

Math.sqrt(16); // retorna 4, suporta qualquer expressão numérica

System.out.println(Math.PI); // 3.14159...

System.out.println(Math.E); //2.17828...

Math.exp(2); // Math.E elevado ao quadrado

Math.Log(Math.E); // retorna 1, suporta expressões numéricas positivas

Math.Log10(10); // retorna 1, suporta expressões numéricas positivas

Math.round(2.4999); // retorna 2, suporta qualquer expressão numérica

Math.round(2.5001); // retorna 3, suporta qualquer expressão numérica

Math.random(); // retorna um double randômico

Java - Escopo de código

- ❑ Variáveis podem ser definidas em qualquer parte do código, a qualquer momento
- ❑ Um bloco em Java é um conjunto de comandos delimitados por { }
- ❑ Cada bloco define o escopo para suas variáveis
- ❑ Variáveis definidas dentro de um bloco de código só “valem”, e podem ser usadas, dentro do bloco em questão, e dentro dos blocos internos ao que a variável foi definida

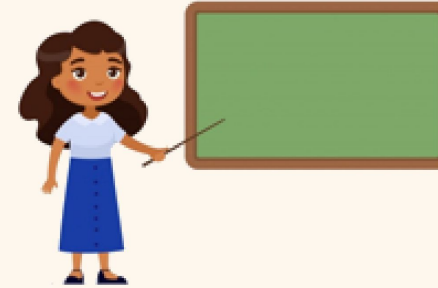
```
public static void main(String[] args) {  
    int a = 2;  
    int b = a + 1;  
    if (a == b) {  
        a = a + 1;  
        double c = 9.22;  
        for(int i = 0; i < b; i++) {  
            c = c + i*2 - b;  
        }  
        i = 0; // i não existe neste escopo  
        System.out.println(c);  
    }  
    c = a + b*8; // c não existe neste escopo }  
}
```

Strings – conceitos preliminares

- ▶ Tipo que, em JAVA, é representado por uma classe nativa da linguagem - String
- ▶ Representa uma cadeia de caracteres
- ▶ Embora seja uma classe, e por conseguinte valores do tipo String sejam objetos, podem ser facilmente manipuladas como um tipo primitivo comum
- ▶ Constantes do tipo String são definidas entre aspas duplas
- ▶ Concatenação de Strings
 - ▶ Usa-se o operador + com, pelo menos, um dos operandos do tipo String
 - ▶ Se dentro da expressão de concatenação existirem operandos de tipos diferentes de String (qualquer tipo do JAVA), estes são convertidos implicitamente para String e a operação de concatenação é completada
 - ▶ O operador += é usado para concatenação quando a variável de destino é do tipo String, independente da expressão do lado direito
- ▶ Ler Strings do teclado: usa-se o comando next() ou nextLine() do Scanner
- ▶ Escrever Strings no console: usa-se o comando de saída normalmente

Strings – conceitos preliminares

Exemplos



```
float f = 3000.22F;  
String a1 = "MARCOS";  
String a2 = a1;  
a2 = a1 + " tem salario de " + f;  
String s1 = "BEATRIZ";  
String s2 = "SILVA";  
final int IDADE = 30;  
String sConcat = s1 + " " + s2 + " tem ";  
sConcat = sConcat + IDADE + " anos e gasta ";  
sConcat += 100.20;  
sConcat += " reais de gasolina";
```

Conversão de tipos primitivos

		Valores possíveis		Valor Padrão	Tamanho	Exemplo
Tipos	Primitivo	Menor	Maior			
Inteiro	byte	-128	127	0	8 bits	byte ex1 = (byte)1;
	short	-32768	32767	0	16 bits	short ex2 = (short)1;
	int	-2.147.483.648	2.147.483.647	0	32 bits	int ex3 = 1;
	long	-9.223.372.036.854.770.000	9.223.372.036.854.770.000	0	64 bits	long ex4 = 1l;
Ponto Flutuante	float	-1,4024E-37	3.40282347E + 38	0	32 bits	float ex5 = 5.50f;
	double	-4,94E-307	1.79769313486231570E + 308	0	64 bits	double ex6 = 10.20d; ou double ex6 = 10.20;
Caractere	char	0	65535	\0	16 bits	char ex7 = 194; ou char ex8 = 'a';
Booleano	boolean	false	true	false	1 bit	boolean ex9 = true;

Conversão de tipos primitivos

boolean

☐ true or false

char



byte



short



int



long



↑
Sign

float

double

Sign Exponent Mantissa

Single Precision (32 bits)



Double Precision (64 bits)



Conversão de tipos primitivos

Compatibilidade de tipos e conversão implícita

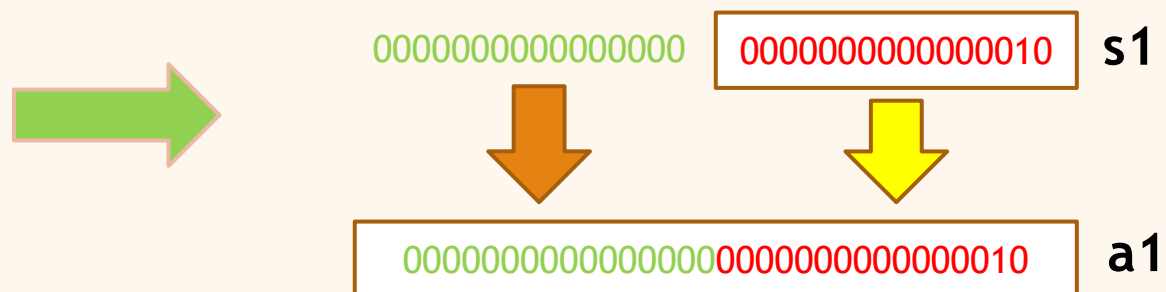
- ▶ JAVA é fortemente tipada!
- ▶ Em uma operação de atribuição, tipos não compatíveis incorrem em erro de compilação
- ▶ O que significa “tipos compatíveis”?
 - ▶ Quando os tipos da variável que recebe a atribuição e a constante/variável/expressão atribuída são os mesmos
 - ▶ Quando os tipos da variável que recebe a atribuição e da constante/variável/expressão atribuída são diferentes mas o tipo desta última “cabe” no tipo da variável – quando isto ocorre, há uma conversão implícita do tipo da constante/variável/expressão para o tipo da variável
- ▶ Uma constante/variável/expressão do tipo boolean só pode ser atribuída a outro boolean
- ▶ O tipo char guarda internamente o código UNICODE (um valor inteiro) de um caractere e portanto aceita atribuição de constantes inteiras dentro da sua faixa (0 a 65535), mas não é recomendável fazer este tipo de operação
- ▶ Para os tipos numéricos, vale a seguinte regra: a atribuição é válida se o tipo da variável estiver, na lista abaixo, à direita do tipo da constante/variável/expressão atribuída

double - float - long - int - short - byte

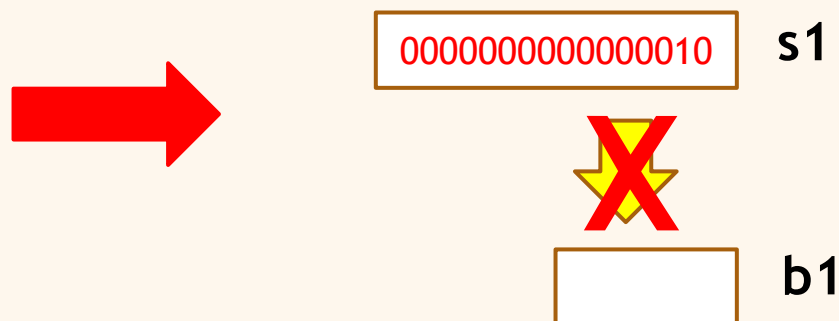
Conversão de tipos primitivos

Compatibilidade de tipos e conversão implícita

```
short s1 = 2;  
int a1;  
a1 = s1; // OK
```



```
short s1 = 2;  
byte b1 = s1; // Não OK
```



A conversão implícita válida é chamada de “widening conversion”, ou conversão de alargamento - onde um valor de tipo mais “curto” é colocado em uma variável de tipo mais “largo”

Conversão de tipos primitivos

conversão explícita - CASTING

- ▶ A conversão implícita se dá quando atribuímos um valor (constante, conteúdo de uma variável ou resultado de uma expressão) de tipo mais “curto” a uma variável de tipo mais “longo”
- ▶ Às vezes, pode ser necessário realizar a operação inversa: atribuir um valor de tipo mais “longo” a uma variável de tipo mais “curto”
- ▶ Neste caso, o programador deve explicitar que a conversão deve ser feita
- ▶ Há risco de perda de informação, já que parte do conteúdo “sobrando” vai ser descartada
- ▶ Não há erros de complicação em CASTING, portanto, o programador deve ter ciência dos riscos de perda de informação, que podem incorrer em funcionamento incorreto do programa e mesmo em erros de execução
- ▶ Quando há perda de informação?
 - ▶ Quando fazemos conversão explícita de tipos decimais para tipos inteiros e os valores envolvidos contém conteúdo decimal significativo
 - ▶ Quando os valores a serem convertidos estiverem fora da faixa suportada pelo tipo mais curto

Conversão de tipos primitivos

Conversão explícita - CASTING

- Como fazer um CASTING?

`<tipo mais curto> <variável> = (<tipo mais curto>) <valor de tipo mais longo>;`

OU

`<variável de tipo mais curto> = (<tipo mais curto>) <valor de tipo mais longo>;`

```
byte b1;  
long ol = 983;  
int a1 = (int)ol;  
short s2 = (short)a1;  
double d1 = 222.00;  
a1 = (int)d1;  
float f1 = -23211.22F;  
s2 = (short)f1;  
b1 = (byte)ol;
```

Exemplo

- Rode o exemplo a seguir. Você consegue explicar o resultado?

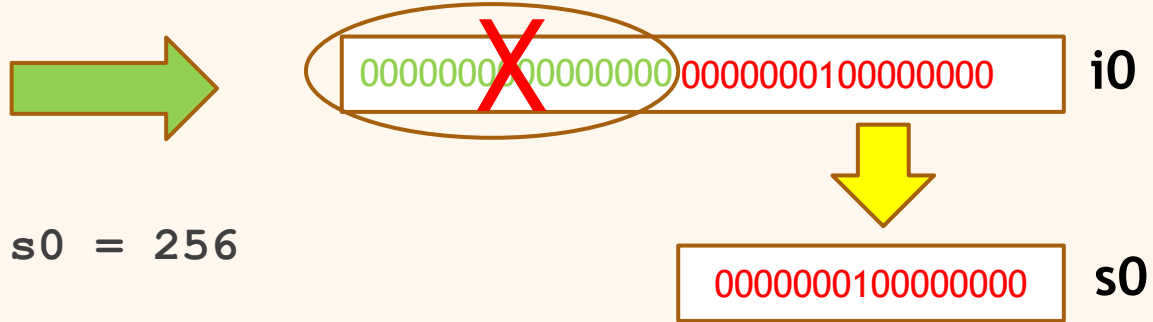
```
public class Exemplo {  
    public static void main(String[] args) {  
        byte b1, b2;  
        short s1 = 256;  
        b1 = (byte) s1;  
        System.out.println("valor de s1: " + s1);  
        System.out.println("valor de b1: " + b1);  
        int i1 = 255;  
        b2 = (byte) i1;  
        System.out.println("valor de i1: " + i1);  
        System.out.println("valor de b2: " + b2);  
    }  
}
```

Conversão de tipos primitivos

Conversão explícita - CASTING

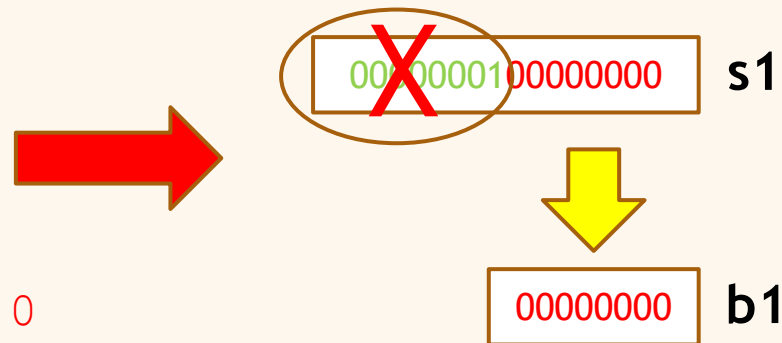
```
int i0 = 256;  
short s0;  
s0 = (short)i0; // s0 = 256
```

(OK)



```
byte b1;  
short s1 = 256;  
b1 = (byte)s1; // b1 = 0
```

(perda)



Na representação binária, 00000000 em um byte é 0!!

A conversão explícita é chamada de “narrowing conversion”, ou conversão de encurtamento - onde um valor de tipo mais “largo” é colocado em uma variável de tipo mais “curto”

Conversão de tipos primitivos

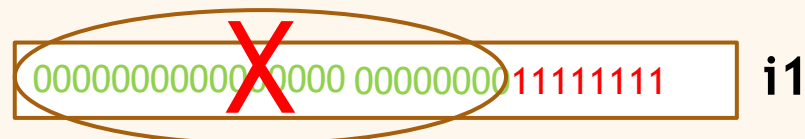
Conversão explícita - CASTING

```
int i1 = 255;
```

```
byte b2;
```

```
b2 = (byte)i1; // b2 = -1
```

(perda)



Na representação binária, 11111111 em um byte é -1!!

```
int i2;
```

```
float f1 = 255.12;
```

```
i2 = (int)f1; // a1 = 255
```

(perda)



0.12



A parte decimal é perdida na conversão, só restando a parte inteira, que é igual a 255



**Slides adaptados a partir
do material produzido pelo
prof. Eduardo Calábria**

egc2@cesar.school





**Maurício da Motta
Braga**



mmb2@cesar.school

